Security

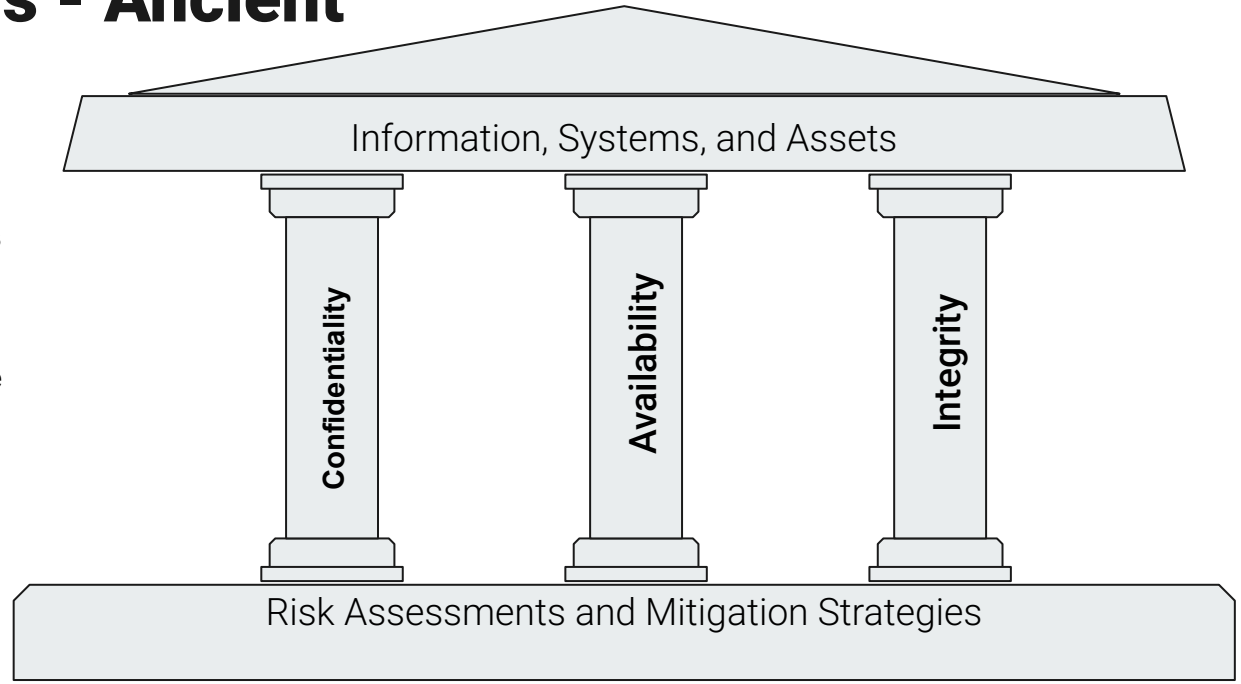
Pillars of Security

Security Pillars - Ancient

Confidentiality - sensitive data does not get in the hands of unauthorized systems or people

Availability- security controls are functioning properly. Systems and people that need the data have it.

Integrity- data is accurate and complete through the data lifecycle

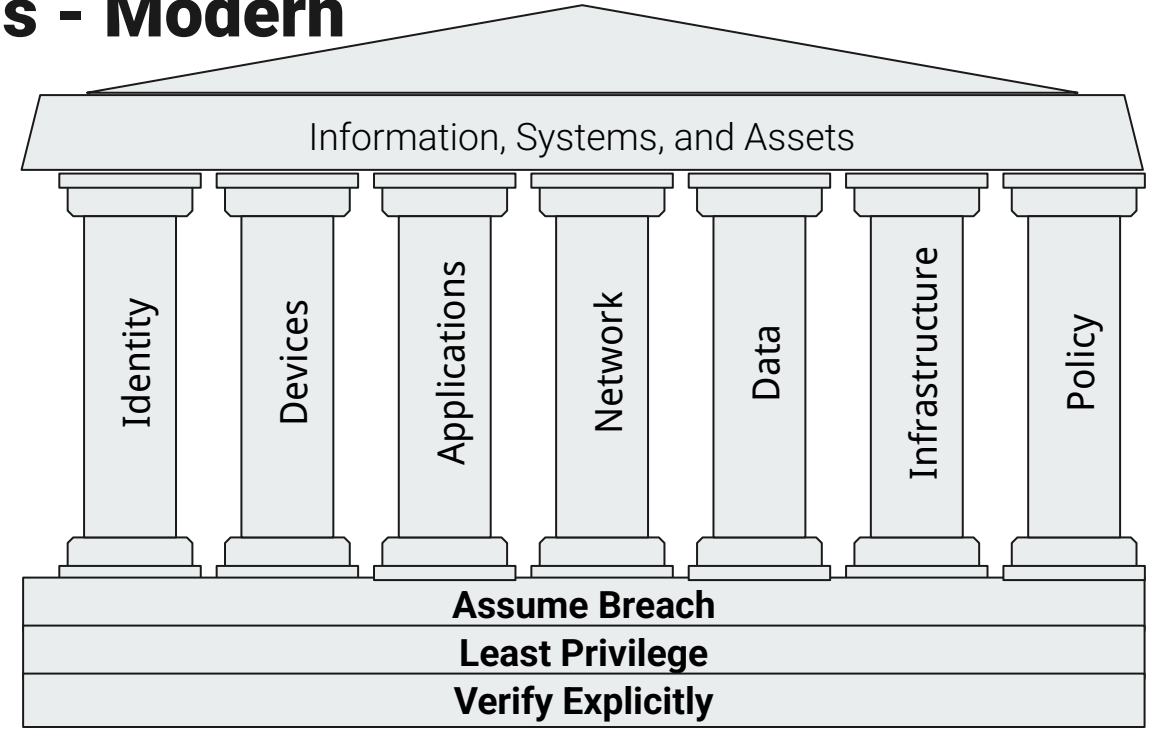


Security Foundations - Modern

Verify Explicitly - Always authenticate and authorize based on all available data points,

Least Privilege- Limit user access with just-in-time and just-enough-access (JIT/JEA), risk-based adaptive policies

Integrity- data is accurate and complete through the data lifecycle



Zero Trust Architecture

Security Design and Architecture

Remember- Security is a Quality Measure

Quality Attributes of Business Systems

- Scalability
- Availability
- Reliability
- Usability
- Performance
- **Security**

Security **Must** be Part of Design and Architecture

Types of Architecture

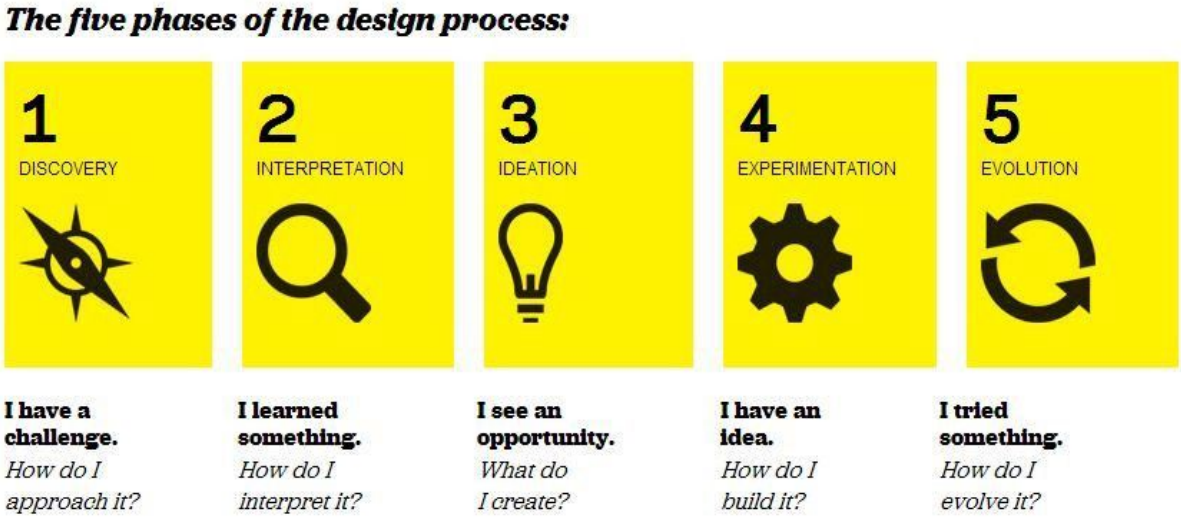
- Enterprise Architecture
- Business Architecture
- Information Architecture
- Software Architecture
- Data Architecture
- Infrastructure Architecture
- **Security Architecture**

But how?

And when?

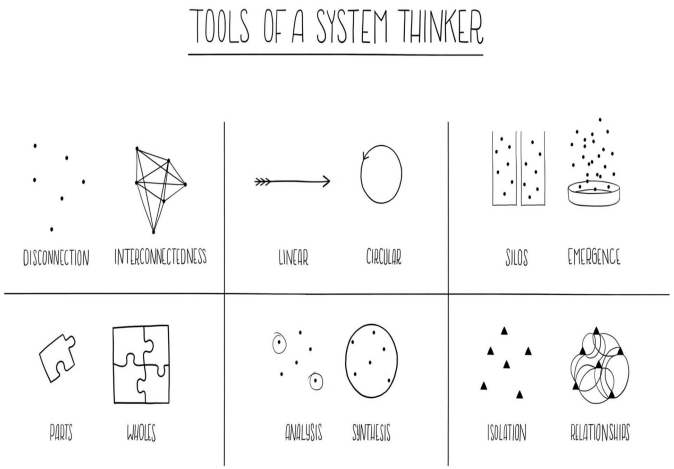
How to Design and Architect for Security

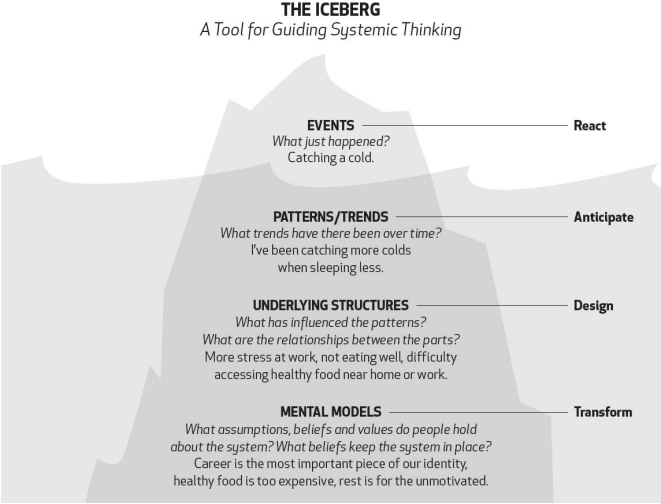
With Design Thinking of Course!



How to Design and Architect for Security

And Systems Thinking Too!





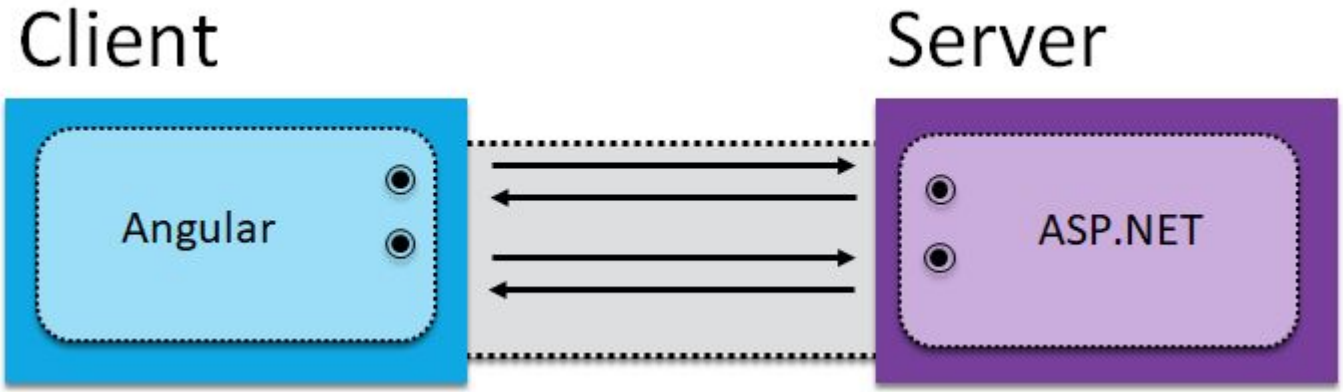
How to Design and Architect for Security

Best Practices for Enterprise Business Systems

1. Implement a strong **identity** foundation
2. Enable **traceability**
3. Apply security at all **layers**
4. **Automate** security best practices
5. Protect data in **transit** and at rest:
6. Keep **people** away from data
7. **Prepare** for security events

Web Application Security

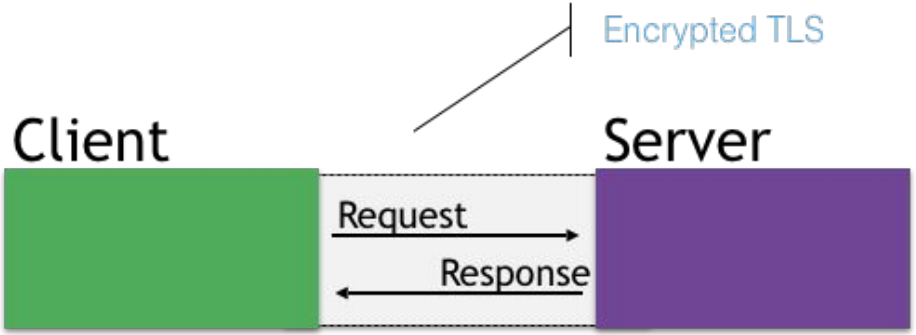
Web Architecture

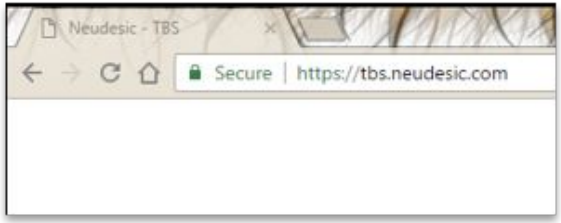


HTTPS

HTTPS protects the conversation between client and server with encryption.

The browser *enforces* security by checking that the certificate is from a trusted authority.





Certificate Authority

Certificate Authorities sell ‘digital trust’ used to encrypt messages from your browser (client) and the server.

Every browser comes pre-configured with a well-known list of **CAs** that it trusts automaticailly.

If the signed cert is from a vendor on that list, you browser trusts it.

Let’s Encrypt

Because encryption and privacy are so important, there is a consortium working to encrypt everything, by default, and for **free**.

More at https://letsencrypt.org/

Platinum

Gold

Silver

Identity, Sessions, Cookies, and Tokens

Implement a strong identity foundation

Authe

-
-
-

Autho

- **what** someone can do
- drive, buy beer, travel
- access university resources, take tests, get credit

How to Design and Architect for Security

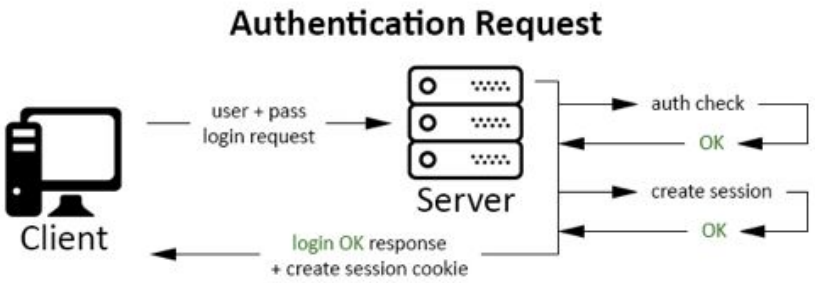
Best Practices for Enterprise Business Systems

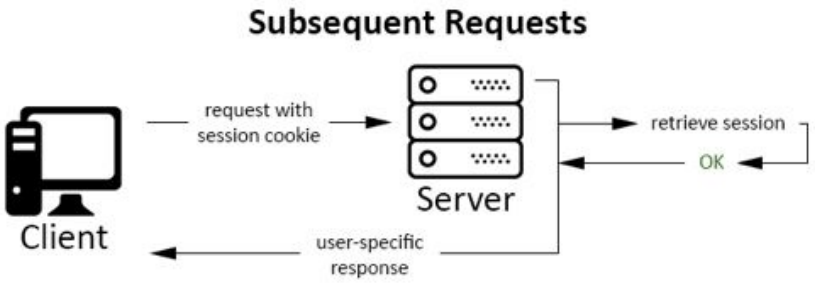
1. Implement a strong **identity** foundation

Securing Web Applications

Sessions

1. User logs in with credentials
2. If authentication passes, session is created
3. 200 OK with session cookie
4. Cookie holds session_id
5. Additional requests sent with session cookie
6. Server looks up session data with session_id

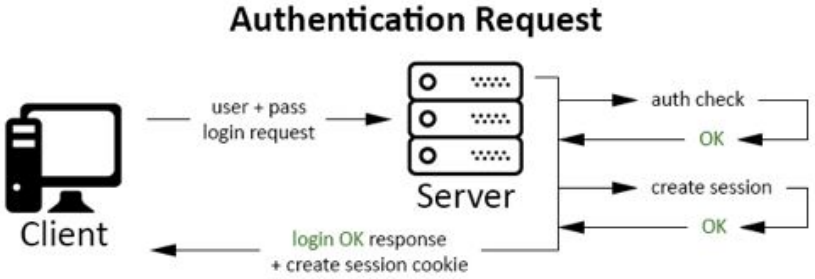


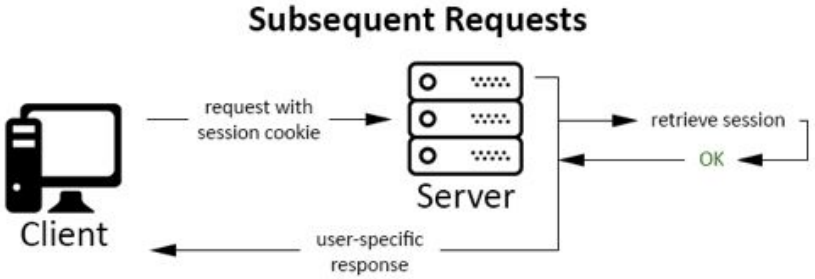


Securing Web Applications

Cookie

1. Stores session metadata, but not details
2. Browser automatically sends cookie for each request in header
3. Used to simulate statefulness
4. Cannot be read by other websites
5. Cookies expire, can be long or short
6. Privacy and scaling issues

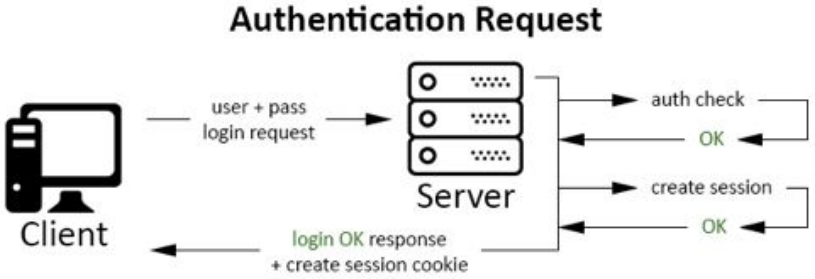


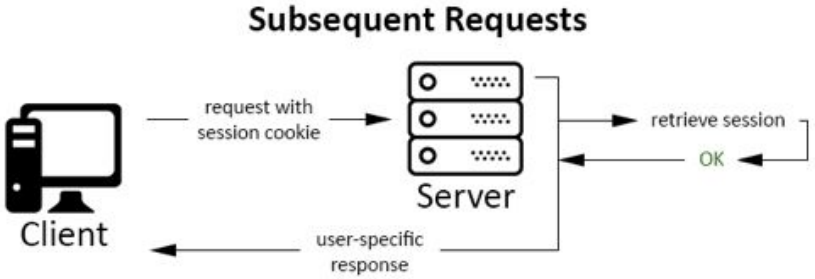


Securing Web Applications

Token

1. Digitally signed, contain more information than session_id
2. Support more clients than just web browsers; use with API and Mobile
3. Generally and genuinely stateless
4. Harder to tamper and highjack

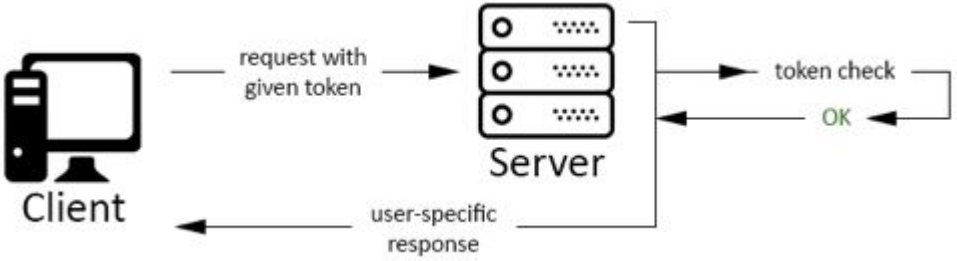




Options for Securing Web Applications

Using Two-Way Cert

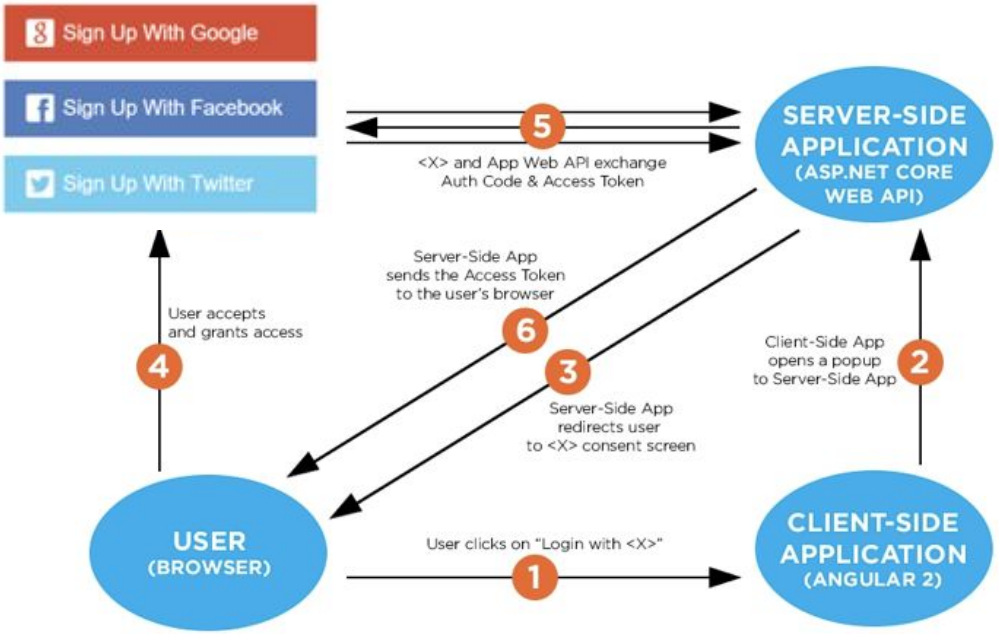
1. Most Secure
2. More difficult to set up
3. Every user gets a unique cert



Options for Securing Web Applications

Using 3rd Party

1. Benefits of Tokens
2. Gets you out of Identity business



OWASP Top 10

#1 - Broken Access Control

What is it?

Access control enforces policy such that users cannot act outside of their intended permissions.

Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits.

Example: not using least privilege, no zero-trust, not deny by default

#2 - Cryptographic Failure

What is it?

Failure to properly encrypt data. This includes data “at rest” and data “in transit”.

For example, passwords, credit card numbers, health records, personal information, and business secrets require extra protection, mainly if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations, e.g., financial data protection such as PCI Data Security Standard (PCI DSS).

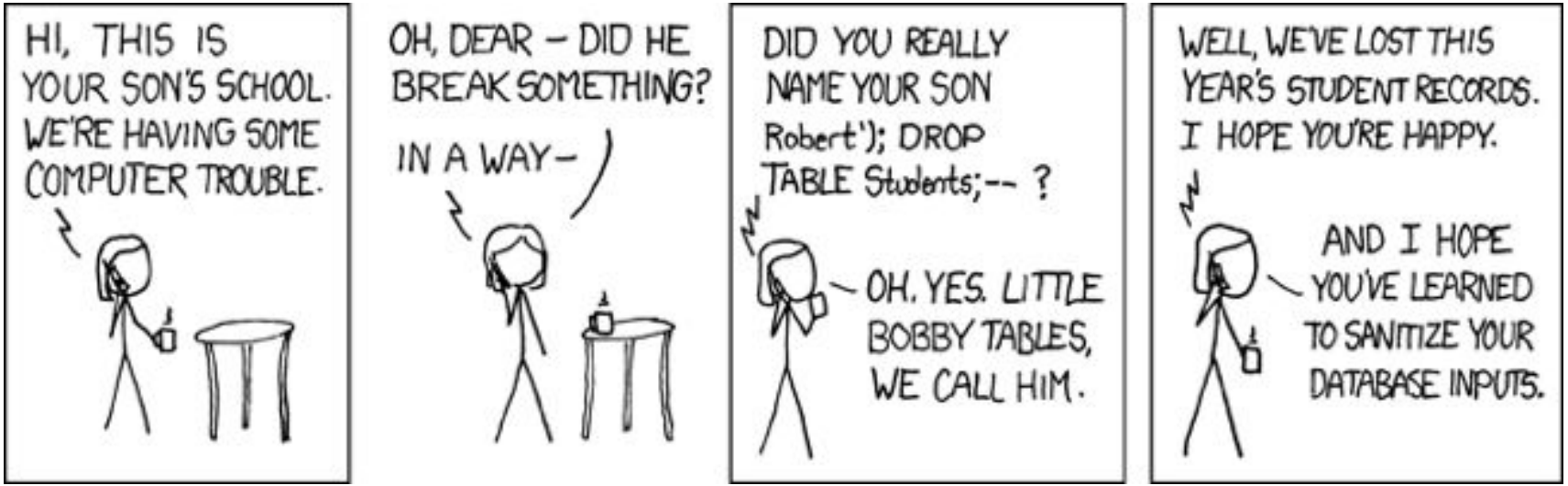
#3 - Injection (f.k.a SQL Injection)

What is it?

An application is vulnerable to attack when:

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
- Hostile data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

Sanitize Your Database Inputs



#4 - Insecure Design

What is it?

Design and architectural flaws that cannot be fixed by simply fixing the implementation. Organizations can prevent with more use of threat modeling, secure design patterns, and reference architectures.

Examples include requirements flaws, poor or naive assumptions, and failure to secure the software development lifecycle.

#5 - Security Misconfiguration

What is it?

An application or infrastructure is not configured properly.

Examples include running sample code in production, not changing default passwords, and opening incorrect network ports.

#6 - Vulnerable and Outdated Components

What is it?

Not patching and updating all components of the software supply chain- e.g. firmware, operating systems, and application dependencies.

#7 - Identification and Authentication Failures

What is it?

Confirmation of the user's identity, authentication, and session management is critical to protect against authentication-related attacks. There may be authentication weaknesses if the application:

- Permits automated attacks such as credential stuffing, where the attacker has a list of valid usernames and passwords.
- Permits brute force or other automated attacks.
- Permits default, weak, or well-known passwords, such as "Password1" or "admin/admin".

#8 - Software and Data Integrity Failures

What is it?

Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations.

An example of this is where an application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs). An insecure CI/CD pipeline can introduce the potential for unauthorized access, malicious code, or system compromise. Not using digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered

#9 - Security Logging and Monitoring Failures

What is it?

Organization must detect, escalate, and respond to active breaches. Without logging and monitoring, breaches cannot be detected. Insufficient logging, detection, monitoring, and active response occurs any time:

- Auditable events, such as logins, failed logins, and high-value transactions, are not logged.
- Warnings and errors generate no, inadequate, or unclear log messages.
- Logs of applications and APIs are not monitored for suspicious activity.
- Logs are only stored locally.

#10 - Server Side Request Forgery

What is it?

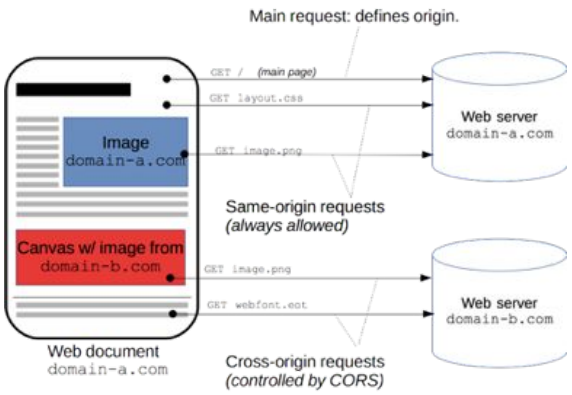
SSRF flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).

As modern web applications provide end-users with convenient features, fetching a URL becomes a common scenario. As a result, the incidence of SSRF is increasing. Also, the severity of SSRF is becoming higher due to cloud services and the complexity of architectures.

Cross-origin Resource Sharing (CORS)

In Lab 6, this is something we need to solve: by default the browser will not allow the client application and server calls to come from two different **origins.**

The solution is to make sure the *server* returns the *client's* origin in the header.



Client Request

```
OPTIONS /
Host: domain-a.com
Origin: domain-b.com
```

Server Response

```
Access-Control-Allow-Origin: domain-b.com
Access-Control-Allow-Methods: PUT, DELETE
```

Thank you.